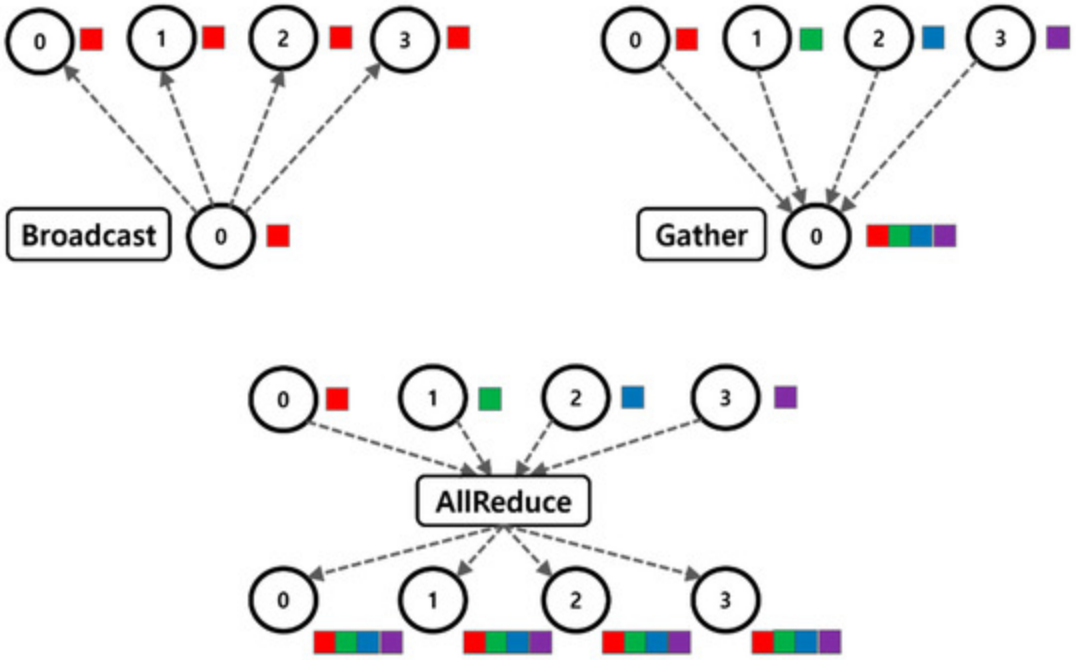
Name:	Krish Sodhi, Marvin Sukumar, Riya Suryawanshi, Anushka Suvarna
UID:	2023300233, 2023300238, 2023300240, 2023300241
Subject:	Distributed Computing
Experiment No.:	11
Problem Statement:	To implement and demonstrate parallel matrix multiplication using the Message Passing Interface (MPI) for distributed computation across multiple processes.
Objectives:	• To understand how matrix multiplication can be parallelized by dividing computation among multiple MPI processes. • To learn how MPI collective operations (Bcast, Scatter, Gather) help distribute matrix data and collect computed results efficiently. • To implement parallel matrix multiplication using mpi4py, where each process multiplies a portion of matrix A with matrix B. • To observe how workload distribution across processes reduces computation time compared to sequential multiplication. • To analyze how MPI ensures synchronization and data consistency during distributed matrix operations.
Theory:	**Message Passing Interface (MPI):** MPI (Message Passing Interface) is a standardized library used for communication between processes in distributed and parallel systems. It enables multiple processes to exchange data efficiently, even when running on different machines. MPI programs run inside a communicator (usually MPI_COMM_WORLD) that contains a group of processes. Each process is assigned a rank, which is used to identify the sender or receiver. MPI supports two major communication types: 1. Point-to-point communication (send/receive) 2. Collective communication (all processes participate) This experiment focuses on collective communication. **MPI Collective Communication** Collective communication operations involve all processes in the communicator.

	They are optimized internally and avoid manual coordination. The three important collective operations used in this experiment are: **1. MPI_Bcast (Broadcast)** 1. One process (root) sends the same data to all other processes. 2. Used when all processes require a common initial value. **2. MPI_Scatter** 1. The root process divides an array and sends different chunks to each process. 2. Useful for distributing work evenly across processes. **3. MPI_Gather** 1. Opposite of scatter. 2. Each process sends its result to the root process. 3. Used when partial results must be combined. **Advantages of MPI Collectives:** 1. Efficient communication (internally optimized) 2. Simple function calls (no manual send/receive) 3. Reduces chances of synchronization errors 4. Essential for parallel tasks such as matrix operations, simulations, data processing



MPI Collective Operations Used in Parallel Processing

Parallel Matrix Multiplication Using MPI: MPI allows us to parallelize matrix multiplication by:

- Dividing the rows of matrix A equally among all processes (using Scatter).
- Broadcasting matrix B to all processes (using Bcast).
- Each process computes a different portion of matrix C (local multiplication)
- Collecting partial results back into the final matrix C (using Gather).

This reduces execution time and demonstrates how distributed memory systems handle large computations.

Algorithm:

Step 1 — Initialize MPI

1. Start MPI environment

2. Get:

    a. rank → process ID
    b. size → number of processes

	Step 2 — Root Process Initializes Data 1. If rank == 0: a. Take user input for broadcast value b. Take a list of integers equal to number of processes for scatter 2. Other ranks set empty placeholders Step 3 — Broadcast the Value 1. Root process calls MPI_Bcast() with the input value 2. All processes receive the same broadcasted value Step 4 — Scatter Input Values 1. Root process prepares a list with one value for each process 2. MPI_Scatter() sends one element to each process 3. Each process receives its portion (one value here) Step 5 — Local Computation on Each Process 1. Each process performs a simple computation: result=scatter_value+broadcast_value Step 6 — Gather Results 1. MPI_Gather() collects all individual results 2. Root process stores them in order of ranks 3. Root prints the final combined result Step 7 — Finalize MPI End the MPI execution environment.

Code:	from mpi4py import MPI
	import numpy as np
	
	comm = MPI.COMM_WORLD
	rank = comm.Get_rank()
	size = comm.Get_size()
	
	# Define matrix size
	N, M, K = 6, 4, 5 # A: N×M B: M×K
	
	if rank == 0:
	  A = np.random.randint(1, 10, size=(N, M), dtype=np.int64)
	  B = np.random.randint(1, 10, size=(M, K), dtype=np.int64)
	else:
	  A = None
	  B = None

```
# Broadcast sizes
sizes = np.array([N, M, K], dtype=np.int64) if rank == 0 else np.empty(3,
dtype=np.int64)
comm.Bcast(sizes, root=0)
N, M, K = map(int, sizes)

# Broadcast matrix B
if rank != 0:
    B = np.empty((M, K), dtype=np.int64)
comm.Bcast(B, root=0)

# Divide A rows
rows_per_rank = np.array(
    [N // size + (1 if r < N % size else 0) for r in range(size)],
    dtype=np.int64
)
displs = np.concatenate(([0], np.cumsum(rows_per_rank[:-1])))
local_rows = rows_per_rank[rank]

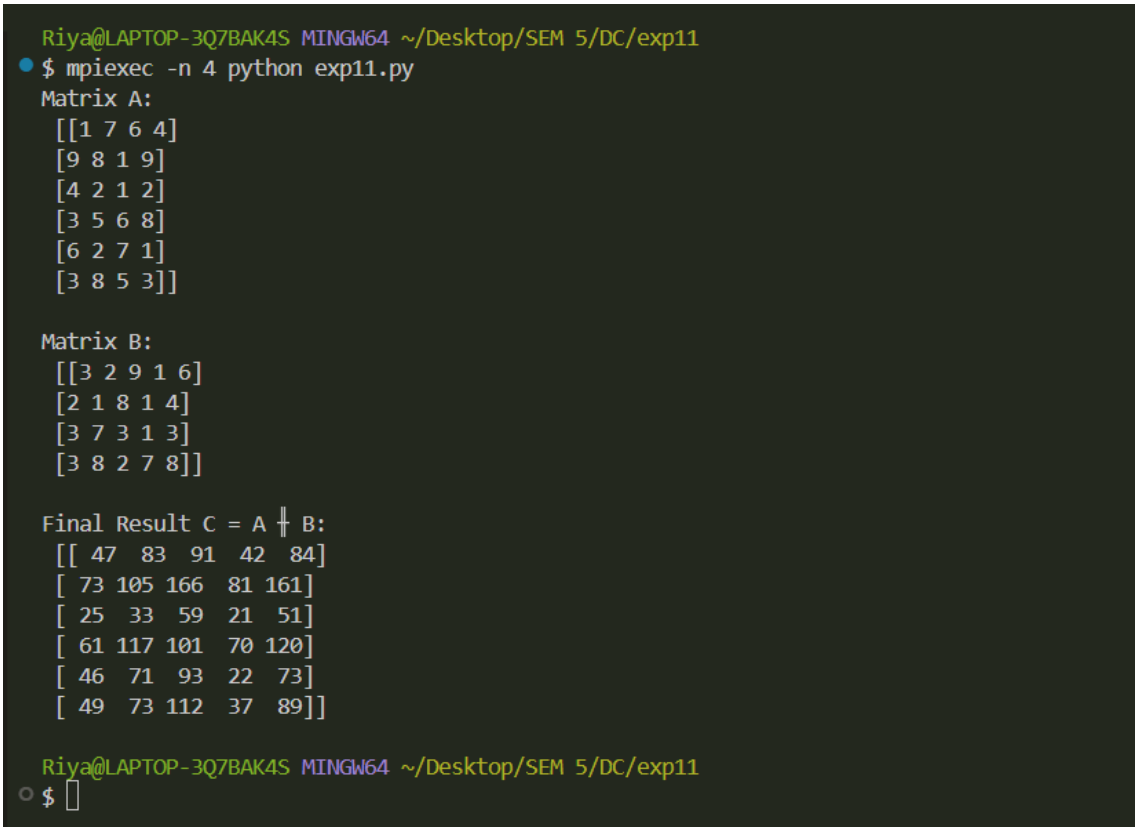
# Scatter matrix A
A_local = np.empty((local_rows, M), dtype=np.int64)
if rank == 0:
    sendbuf = [A, rows_per_rank * M, displs * M, MPI.INT64_T]
else:
    sendbuf = None

comm.Scatterv(sendbuf, [A_local, local_rows * M, MPI.INT64_T], root=0)

# Local multiplication
C_local = A_local.dot(B)

# Gather matrix C
if rank == 0:
    C = np.empty((N, K), dtype=np.int64)
    comm.Gatherv(
        [C_local, local_rows * K, MPI.INT64_T],
        [C, rows_per_rank * K, displs * K, MPI.INT64_T],
        root=0
    )

    print("Matrix A:\n", A)
    print("\nMatrix B:\n", B)
    print("\nFinal Result C = A × B:\n", C)
```

	else: comm.Gatherv([C_local, local_rows * K, MPI.INT64_T], None, root=0)
Output:	
Conclusion:	The experiment successfully demonstrated Parallel Matrix Multiplication using MPI. By distributing rows of matrix A across multiple processes and broadcasting matrix B, the multiplication was performed efficiently in parallel. MPI functions such as Bcast, Scatterv, and Gatherv enabled proper synchronization and data exchange between processes. The results confirmed correct parallel computation and showed how MPI can speed up heavy matrix operations, proving its usefulness in scientific and engineering applications.